King Saud University
College of Computer and Information Sciences
Department of Information Technology

Analyzing LinkedIn Job Postings

IT462 Big Data Systems

Final Report

Prepared by

Students	*ID*	*Email Addresses*
Ghadeer Khalid Alnuwaysir	444200420	444200420@student.ksu.edu.sa
Jana Alruzuq	444201157	444201157@student.ksu.edu.sa
Rana Abdulmohsen Albridi	444201094	444201094@student.ksu.edu.sa
Ghena Almogayad	444203140	444203140@student.ksu.edu.sa

Supervised by:

Dr. Afshan Jafri

Date of submission:

Second Semester 1447 H

Table of Contents

Table of Figures

Table Of Tables

1. Introduction

1.1 Background and Context

The labor market has undergone significant transformation in recent years, driven by the rapid digitalization of recruitment processes and the widespread adoption of online job platforms. Websites such as LinkedIn, Indeed, and Glassdoor now serve as primary channels for job advertising, generating millions of structured and unstructured records that reflect real-time labor market dynamics. Understanding the dynamics of online job postings has become an essential tool for capturing real-time labor market trends) Tzimas(2024 ،.

This shift toward digital recruitment has created an unprecedented opportunity for large-scale data analysis. Analyzing job postings on platforms like LinkedIn can be of great value to human resource managers, policymakers, and job seekers to understand categories and the prevalence of the modern job market. However, the sheer volume of available data, often reaching hundreds of thousands of records, necessitates scalable big data frameworks capable of handling complex, multi-typed datasets efficiently.

From a social and economic perspective, understanding salary structures is critical. Building an accurate and reliable predictive model holds significant implications not only for individual education and career planning but also for social policy and talent development strategy. Yet salary information in job postings remains inconsistent and self-reported, making it difficult to derive reliable compensation benchmarks without systematic analytical methods)Lasi(2014 ،.

1.2 Problem Statement

This project aims to analyze and predict salary levels in online job postings using large-scale data analytics. Specifically, it seeks to identify and model the key factors influencing salary variations, such as job title, employment type, geographic location, and experience level, and to build a regression model for salary estimation. Using the LinkedIn Job Postings dataset (2023–2024) (Kolasa, n.d.) and leveraging Apache Spark for scalable processing, the project addresses the challenge of handling high-volume, multi-structured data. The outcomes of this project are

valuable for multiple stakeholders, including job seekers seeking reliable compensation benchmarks, recruiters aiming to design competitive salary packages, academic institutions aligning curricula with labor market demand, and policymakers monitoring workforce trends.

1.3 Objectives

- To collect, clean, and preprocess the LinkedIn Job Postings dataset (2023–2024) from Kaggle, including handling missing values, duplicates, and irrelevant features.
- To explore key labor market patterns, such as salary distributions across location, employment type, and job titles, using Apache Spark RDD operations and Spark SQL.
- To perform feature engineering, including extracting meaningful attributes and applying TF-IDF and salary transformations.
- To build and evaluate regression models (e.g., Linear Regression, Random Forest, Gradient Boosted Trees) for salary prediction using Spark MLlib.
- To interpret model results and communicate actionable insights for stakeholders such as job seekers, recruiters, and policymakers.

1.4 Scope and Limitations

In Scope:

- The primary dataset used is postings.csv from the LinkedIn Job Postings dataset (2023–2024), sourced from Kaggle.
- Analysis covers preprocessing, RDD-based exploration, SQL queries, and supervised machine learning (regression).
- All processing is performed using Apache Spark (RDDs, DataFrames, Spark SQL, MLlib) in a distributed computing environment.
- The target prediction task is annual salary estimation based on structured job-level features.

Out of Scope:

- Real-time data ingestion or deployment of the predictive model as a production service.
- Use of secondary dataset files (e.g., companies.csv, company_industries.csv) beyond the main postings file.

- Deep hyperparameter tuning via automated search (e.g., CrossValidator with ParamGridBuilder) due to computational constraints.
- Generalization to job markets outside the United States or beyond the 2023–2024 time period.
- Use of advanced NLP embeddings (e.g., BERT, Word2Vec) for job description processing.

2 Data Selection and Understanding

2.1 Dataset Description

This dataset gives us a detailed snapshot of the current job market by looking at over 124,000 real job postings collected from LinkedIn during 2023 and 2024. For every single job, it includes a variety of useful details like the job title, a full text description of the role, salary ranges, where the job is located, and the type of work (like whether it is full-time, contract, or remote). We chose this dataset because it provides a massive amount of real-world information that will allow us to explore hiring trends, see which locations or roles pay the most, and understand exactly what companies are looking for right now.

2.2 Dataset source and size

- **Dataset Name:** LinkedIn Job Postings (2023-2024)
- **Domain:** Labor Market Analytics & Recruitment
- **Source URL:** https://www.kaggle.com/datasets/arshkon/linkedin-job-postings\
- **Platform:** Kaggle
- **Number of rows:** 124,000+ job postings
- **File size:** 516.84 MB (CSV format)
- **Number of files:** 1 primary main CSV file and 10 secondary CSV files
- **Structure:** Multi-table relational dataset

The project relies only on the main `postings.csv` file. Although the original Kaggle dataset contains multiple related files, such as `companies.csv` &

`company_industries.csv` this study uses a single structured table that contains all necessary job-level information for analysis and modeling. With a file size of 516.84 MB and 124,000+ job postings, the dataset is considered large-scale, making it suitable for big data analysis.

2.3 Structure and Schema

2.3.1 Main attributes and types

Table 1: Main attributes and types

Attribute	*Description*	*Type*
job_id	Unique job identifier	Categorical (Identifier)
company_id	Company identifier	Categorical (Identifier)
title	Job title	Text
description	Job description	Text
max_salary	Maximum salary	Numeric
med_salary	Median salary	Numeric
min_salary	Minimum salary	Numeric
pay_period	Salary period (Hourly, Monthly, Yearly)	Categorical
formatted_work_type	Type of employment	Categorical
location	Job location	Categorical

Based on the dataset's structure, the **job_id** serves as our primary key because it is a unique number created by LinkedIn that guarantees no two job postings are exactly the same.
The **company_id** acts as a foreign key, which links the jobs in our main table to the specific businesses listed in the separate companies' dataset. Finally, if we didn't have those official ID numbers, we could use the **job_posting_url** as a natural key since every job has its own unique web address or even a combination of the job **title**, **company_id**, and **location** to pinpoint a specific real-world opening.

2.4 Initial Data Quality Observations

Looking over the dataset summary, we noticed a few data quality issues that will need to be cleaned up before doing the main analysis:

- **Missing Data:** A huge chunk of the salary information is missing. For example, the max_salary column is blank about 71% of the time. We'll need to either filter these out or find a way to handle the missing numbers.
- **Weird Salary Numbers:** Some of the salaries are crazy high, with maximums listed in the millions (like 120m). These are probably typos by whoever posted the job or the wrong currency, so we'll need to remove those extreme outliers.
- **Mixed Pay Periods:** The salaries are listed differently, some are hourly, some are monthly, and some are yearly. We won't be able to compare them fairly until we do some math to convert them all into a standard yearly salary.
- **Duplicate Jobs:** The dataset has over 124,000 rows, but only about 107,000 unique job descriptions. This means there are some duplicate postings. This probably happens when a company posts the exact same job in several different cities.

Overall Quality: Even with these issues, the dataset has a perfect 10.0 usability score on Kaggle. Other users say it's pretty clean and well-documented, so it’s still a really good dataset to work with.

2.5 Ethical and Legal Considerations

When working with this data, there are a few ethical and legal points to keep in mind:

- **Giving Credit (Licensing):** The dataset is shared under a CC BY-SA 4.0 license. This simply means we legally have to give credit to the original creator (Arsh Koneru and his team) in our project, and if we share our modified data, we have to use the same license.
- **Scraped Data:** The data was collected using a web scraper bot. Even though the information was public, scraping usually goes against LinkedIn’s terms of service. It's important to just be honest about how the data was gathered.
- **Privacy:** Because these are public job postings for companies, there isn't really any secret personal data to worry about. However, sometimes a recruiter's actual name or email gets left in the job description. Ethically, we should ignore that info and not use it to contact anyone.

- **Bias in the Data:** This data only represents jobs posted on LinkedIn. LinkedIn is heavily focused on corporate, tech, and office jobs, so it doesn't really show trade, retail, or blue-collar jobs. We need to remember that any conclusions we make will only reflect the "LinkedIn job market," not the entire economy.

3 Data Preprocessing

Before conducting large-scale analysis and salary prediction, we performed a structured preprocessing pipeline consisting of data cleaning, data reduction, and data transformation. We first removed incomplete records and handled missing values to ensure dataset reliability. We then reduced dimensionality through feature selection and created a machine-learning-specific dataset by filtering and trimming salary values. Finally, we engineered additional features, including logarithmic transformations, temporal attributes, location-based features, and standardized categorical variables. These steps prepared the dataset for efficient analysis and modeling.

3.1 Data Cleaning

We began the preprocessing phase by performing data cleaning to ensure the dataset was consistent and reliable for further analysis. Initially, the dataset contained 123,849 rows, as shown in Figure 1.

First, we checked for duplicate records using the job_id attribute as a unique identifier. We applied duplicate detection and found no duplicate records, so no rows were removed at this stage. Next, we handled missing critical values. We removed rows that were missing essential attributes required for analysis, such as job_id, title, and company_name. This step resulted in the removal of 1,719 rows, reducing the dataset to 122,130 rows. We then addressed missing numerical values in selected engagement-related columns. Specifically, we replaced null values in the views, applies, and remote_allowed columns with 0.0. This ensured that these fields could be used safely in later aggregations and modeling without introducing null-related errors.

Finally, after completing all cleaning steps, the dataset contained 122,130 clean rows, as shown in Figure 1 and Table 2. No additional rows were removed due to errors or outliers during this phase.

Table 2: Data Cleaning Table

Issue	*Before*	*After*
Total row count	*123,849*	*122,130*
Rows with missing critical data	*1,719*	*0*
Duplicate records	*0*	*0*
Rows with errors & outliers	*0*	*0*
Missing values (views, applies, remote_allowed)	*Null / Blank*	*Imputed with 0.0*

```
==================================================
          DATA CLEANING STATISTICS REPORT
==================================================
Initial rows loaded:                  123849
Duplicate records removed:            0
Rows dropped (missing critical):      1719
Rows dropped (errors & outliers):     0
Missing values imputed with 0.0:      views, applies, remote_allowed
Final clean row count:                122130
==================================================


scala>
```

Figure 1: Data Cleaning Statistics Report

3.2 Data Reduction

After completing data cleaning, we performed data reduction to decrease dimensionality and prepare a specialized dataset for salary analysis and machine learning. Before reduction, the cleaned dataset contained 122,129 rows and 33 columns, as shown in Figure 2.

First, we performed feature selection. We removed 13 unnecessary columns, reducing the number of columns from 33 to 20, while keeping all 122,129 rows unchanged. This step reduced dimensionality without affecting the number of records. Next, we created a machine-learning-specific dataset by applying a salary filter. Since salary prediction requires labeled data, we kept

only rows with available salary values. This step reduced the dataset from 122,129 rows to 35,562 rows, removing 86,567 rows (70.88%). We then applied a salary trimming step to remove extreme salary values. After trimming, the dataset was reduced to 35,118 rows, removing 444 additional rows. The resulting salary distribution had a minimum value of 10,000 and a maximum value of 960,000, with a mean salary of approximately 96,688. Finally, we performed an aggregation step by state. This produced a summarized dataset containing 297 state-level rows, including metrics such as total postings, total applications, distinct companies, average salary, and median salary. The top 15 states are displayed in the reduction output (Figure 2).

Through these reduction steps, we successfully decreased dimensionality, created a focused salary dataset for machine learning, and generated aggregated analytical outputs while maintaining meaningful information.

```
=== DATA REDUCTION (BEFORE) ===
Rows : 122129
Cols : 33

--- STEP 1: FEATURE SELECTION ---
Rows: 122129 → 122129 (removed: 0)
Cols: 33 → 20 (removed: 13)

--- STEP 2: SALARY FILTER (ML ONLY) ---
Rows: 122129 → 35562 (removed: 86567, 70.88%)

--- STEP 3: SALARY TRIM (ML ONLY) ---
Rows: 35562 → 35118 (removed: 444)
+-------+-----------------+
|summary|normalized_salary|
+-------+-----------------+
|count  |35118            |
|min    |10000.0          |
|25%    |52000.0          |
|50%    |82500.0          |
|75%    |125000.0         |
|max    |960000.0         |
|mean   |96687.59448160486|
+-------+-----------------+


--- STEP 4: AGGREGATION OUTPUTS ---
By state rows    : 297

Top 15 states:
+-------------+--------------+-------------+------------------+--------------+----------+-------------+
|state        |total_postings|total_applies|distinct_companies|salary_postings|avg_salary|median_salary|
+-------------+--------------+-------------+------------------+--------------+----------+-------------+
|United States|11647         |107705.0     |5415              |3857          |122987.19 |111748.0     |
|CA           |11347         |19655.0      |4008              |5731          |330422.79 |92481.5      |
|TX           |10155         |14311.0      |3301              |1948          |658554.97 |77500.0      |
|NY           |5955          |11578.0      |2285              |2966          |143200.1  |87500.0      |
|FL           |5818          |5623.0       |2227              |1366          |249975.21 |66500.0      |
|NC           |4905          |4360.0       |1371              |832           |207671.64 |78000.0      |
|IL           |4428          |6556.0       |1879              |1074          |93510.04  |79040.0      |
|PA           |4079          |3919.0       |1493              |846           |88171.94  |70948.8      |
|VA           |3632          |4937.0       |1301              |805           |98780.38  |86445.5      |
|MA           |3460          |4714.0       |1412              |874           |104482.21 |88649.6      |
|GA           |3385          |4444.0       |1474              |761           |92138.11  |82500.0      |
|OH           |3382          |2607.0       |1348              |712           |83639.27  |67995.2      |
|NJ           |3246          |5567.0       |1303              |877           |97884.57  |87500.0      |
|MI           |2821          |2075.0       |1021              |532           |73194.61  |65000.0      |
|WA           |2673          |4356.0       |1072              |1343          |183473.38 |91936.0      |
+-------------+--------------+-------------+------------------+--------------+----------+-------------+
```

Figure 2: Data Reduction Steps

3.3 Data Transformation

After completing cleaning and reduction, we performed data transformation to prepare the dataset for analytical processing and machine learning. The transformation phase resulted in a dataset containing 122,129 rows and 35 columns, as shown in Figure 3.

First, we created cleaned text fields to ensure consistency. We generated title_clean and company_clean columns by standardizing the original title and company name fields. We also standardized employment type by creating the work_type_std column. Next, we engineered several numerical features to improve analytical capability. We created logarithmic transformations for engagement and salary-related variables, including log_views, log_applies, and log_salary. These transformations help stabilize skewed distributions while preserving information. We also extracted time-based features from the listing timestamp. Specifically, we created listed_month, listed_dow (day of week), and listed_hour. These features allow analysis of temporal posting patterns. In addition, we engineered title-based features by computing title_len (number of characters) and title_words (number of words in the title). These features capture structural characteristics of job titles. Location-related transformations were also applied. We generated location_clean and extracted a state column for geographic analysis. The pay period was standardized into pay_period_std, and the existing formatted_experience_level attribute was preserved for analysis. Finally, we created a binary feature is_remote to represent remote work eligibility as 0 or 1.

Through these transformation steps, we enriched the dataset with additional structured features while maintaining consistency and analytical readiness.

```
=== TRANSFORMATION OUTPUT ===
Rows: 122129
Cols: 35
snap: org.apache.spark.sql.Dataset[org.apache.spark.sql.Row] = [job_id: bigint, company_name: string ... 33 more
 fields]

--- TABLE 1: Job Info (10 rows) ---
+----------+--------------------------------+----------------------------------------+------------+--------
-+
|job_id    |title_clean                     |company_clean                           |work_type_std|is_remot
e|
+----------+--------------------------------+----------------------------------------+------------+--------
-+
|9615617   |inside customer service associate|glastender, inc.                       |FULL-TIME    |0
 |
|935210241 |transactional attorney          |einbinder & dunn llp                    |FULL-TIME    |0
 |
|2558399667|validation engineer, labware lims|i.t. solutions, inc.                   |CONTRACT     |0
 |
|2914254129|director of operations          |eastern indiana works                   |FULL-TIME    |0
 |
|2920450495|service coordinator             |grunwald mechanical contractors & engineers|FULL-TIME    |0
 |
|2974397965|marketing coordinator           |lynx systems                            |FULL-TIME    |0
 |
|3018278978|seasonal office administrator   |aston carter                            |CONTRACT     |0
 |
|3045980831|project engineer                |armstrong builders llc                  |FULL-TIME    |0
 |
|3075721793|architect/project manager       |asa architects                          |FULL-TIME    |0
 |
|3251984218|sap btp                         |sls solutions                           |CONTRACT     |0
 |
+----------+--------------------------------+----------------------------------------+------------+--------
```

Figure 3: Transformed Job Information Features

```
--- TABLE 2: Salary Info (10 rows) ---
+----------+----------+----------+-----------------+------------------+
|job_id    |min_salary|max_salary|normalized_salary|log_salary        |
+----------+----------+----------+-----------------+------------------+
|9615617   |NULL      |NULL      |NULL             |NULL              |
|935210241 |NULL      |NULL      |NULL             |NULL              |
|2558399667|60.0      |70.0      |135200.0         |11.814517839013352|
|2914254129|55000.0   |60000.0   |57500.0          |10.959557617938563|
|2920450495|55000.0   |75000.0   |65000.0          |11.082157933374816|
|2974397965|75000.0   |85000.0   |80000.0          |11.289794413577894|
|3018278978|NULL      |NULL      |NULL             |NULL              |
|3045980831|60000.0   |90000.0   |75000.0          |11.225256725762893|
|3075721793|50000.0   |60000.0   |55000.0          |10.915106645867503|
|3251984218|NULL      |NULL      |NULL             |NULL              |
+----------+----------+----------+-----------------+------------------+


--- TABLE 3: Location Info (10 rows) ---
+----------+--------------+-------------+-------------+-------------------------+
|job_id    |location_clean|state        |pay_period_std|formatted_experience_level|
+----------+--------------+-------------+-------------+-------------------------+
|9615617   |saginaw, mi   |mi           |NULL         |NULL                     |
|935210241 |new york, ny  |ny           |NULL         |NULL                     |
|2558399667|foster city, ca|ca          |HOURLY       |Mid-Senior level         |
|2914254129|muncie, in    |in           |YEARLY       |NULL                     |
|2920450495|omaha, ne     |ne           |YEARLY       |NULL                     |
|2974397965|richardson, tx|tx           |YEARLY       |NULL                     |
|3018278978|dayton, or    |or           |NULL         |Entry level              |
|3045980831|kailua-kona, hi|hi          |YEARLY       |NULL                     |
|3075721793|las cruces, nm|nm           |YEARLY       |NULL                     |
|3251984218|united states |united states|NULL         |NULL                     |
+----------+--------------+-------------+-------------+-------------------------+
```

Figure 4: Transformed Salary and Location Features

```
--- TABLE 4: Engagement (10 rows) ---
+----------+-----+------------------+-------+------------------+
|job_id    |views|log_views         |applies|log_applies       |
+----------+-----+------------------+-------+------------------+
|9615617   |4.0  |1.6094379124341003|1.0    |0.6931471805599453|
|935210241 |1.0  |0.6931471805599453|0.0    |0.0               |
|2558399667|0.0  |0.0               |0.0    |0.0               |
|2914254129|10.0 |2.3978952727983707|0.0    |0.0               |
|2920450495|4.0  |1.6094379124341003|0.0    |0.0               |
|2974397965|3.0  |1.3862943611198906|0.0    |0.0               |
|3018278978|279.0|5.634789603169249 |2.0    |1.0986122886681096|
|3045980831|4.0  |1.6094379124341003|0.0    |0.0               |
|3075721793|0.0  |0.0               |0.0    |0.0               |
|3251984218|89.0 |4.499809670330265 |8.0    |2.1972245773362196|
+----------+-----+------------------+-------+------------------+


--- TABLE 5: Time & Title (10 rows) ---
+----------+------------+---------+-----------+---------+-----------+
|job_id    |listed_month|listed_dow|listed_hour|title_len|title_words|
+----------+------------+---------+-----------+---------+-----------+
|9615617   |4           |2        |22         |33       |4          |
|935210241 |4           |5        |23         |22       |2          |
|2558399667|4           |6        |1          |33       |4          |
|2914254129|4           |3        |15         |22       |3          |
|2920450495|4           |5        |23         |19       |2          |
|2974397965|4           |5        |23         |21       |2          |
|3018278978|4           |7        |12         |29       |3          |
|3045980831|4           |6        |2          |16       |2          |
|3075721793|4           |6        |4          |25       |2          |
|3251984218|4           |2        |23         |7        |2          |
+----------+------------+---------+-----------+---------+-----------+
```

Figure 5: Transformed Engagement and Time Features

Figure 6 below shows the final schema of the preprocessed dataset, including updated data types and newly engineered features. The dataset is now structured and analysis ready.

```
=== SCHEMA: FINAL PREPROCESSED DATASET ===
root
 |-- job_id: long (nullable = true)
 |-- company_name: string (nullable = true)
 |-- title: string (nullable = true)
 |-- max_salary: double (nullable = true)
 |-- pay_period: string (nullable = true)
 |-- location: string (nullable = true)
 |-- views: double (nullable = true)
 |-- med_salary: double (nullable = true)
 |-- min_salary: double (nullable = true)
 |-- applies: double (nullable = true)
 |-- original_listed_time: double (nullable = true)
 |-- remote_allowed: double (nullable = true)
 |-- expiry: double (nullable = true)
 |-- formatted_experience_level: string (nullable = true)
 |-- work_type: string (nullable = true)
 |-- normalized_salary: double (nullable = true)
 |-- listed_time_ts: timestamp (nullable = true)
 |-- closed_time_ts: timestamp (nullable = true)
 |-- work_type_std: string (nullable = true)
 |-- currency_std: string (nullable = true)
 |-- title_clean: string (nullable = true)
 |-- company_clean: string (nullable = true)
 |-- location_clean: string (nullable = true)
 |-- pay_period_std: string (nullable = true)
 |-- is_remote: integer (nullable = true)
 |-- log_views: double (nullable = true)
 |-- log_applies: double (nullable = true)
 |-- listed_date: date (nullable = true)
 |-- listed_month: integer (nullable = true)
 |-- listed_dow: integer (nullable = true)
 |-- listed_hour: integer (nullable = true)
 |-- state: string (nullable = true)
 |-- title_len: integer (nullable = true)
 |-- title_words: integer (nullable = true)
 |-- log_salary: double (nullable = true)
```

Figure 6: Final Preprocessed Dataset Schema

Due to file size limitations, the raw and preprocessed datasets were not uploaded to GitHub. Instead, they are provided via Google Drive. The GitHub repository contains the full preprocessing code, while the dataset files are accessible through the Drive link (see Appendix A).

3.4 Summary of Preprocessing Pipeline

Since the project utilizes a single comprehensive dataset, no data integration was required. The preprocessing pipeline was therefore streamlined into three core phases: **cleaning** to remove incomplete records and address missing values, **reduction** to eliminate unnecessary columns and isolate a focused subset of valid salary data, and **transformation** to engineer new mathematical, temporal, and geographic features.

4 RDD Operations

4.1 RDD Design

The RDD was built from the preprocessed data, where each record is of type RDD[Row]. Across the transformations, the data was reshaped into key-value pairs based on the analysis goal. The filtered subset in Task 1 was stored as RDD[Row] containing only full-time postings with known salary values. In Task 2, the data became RDD[(jobId, title, location, salary, bucket)] so each posting could be assigned to a salary tier. In Task 3, the titles were transformed into RDD[(word, 1)] so words from job titles could be counted and ranked by frequency. In Task 4, the data was mapped into RDD[(location, (applies, salary, salaryCount, postingCount))] and then into a summarized location-level structure containing total applies, average salary, and posting volume. In Task 5, it was reshaped into RDD[(avgSalary, (location, applies, postingCount))] so locations could be ranked by salary.

RDDs were chosen over DataFrames and SQL because they give more control over how data is structured and processed. They allowed us to apply custom filtering logic, compute percentile thresholds and use them directly in the transformation, and combine multiple values in one aggregation step.

4.2 Key RDD Analysis Blocks

5 RDD analysis blocks were implemented, Each block applies a combination of RDD operations to produce a meaningful result.

1. *Task 1: Salary-focused filtering of job postings*
   - Input: Full preprocessed RDD (122,129 rows).
   - Operations: filter, cache, map, take, first
   - Goal: Keep only full-time job postings that have a salary value, so later salary analysis is based on comparable records.

```
val rdd_fulltime_salary = rdd.filter { row =>
  val workType = Option(row.getAs[String]("work_type_std")).getOrElse("").trim.toUpperCase
  val salary   = row.getAs[Any]("normalized_salary")
  workType == "FULL-TIME" && salary != null
}.cache()

val task1Count = rdd_fulltime_salary.count()
println(s"Rows after filtering: $task1Count")

println("\nSample output (5 rows):")
{
  rdd_fulltime_salary
    .map(row => (
      row.getAs[Any]("job_id"),
      row.getAs[String]("title_clean"),
      row.getAs[String]("work_type_std"),
      row.getAs[Any]("normalized_salary")
    ))
    .take(5)
    .foreach { case (id, title, workType, salary) =>
      println(f"  job_id=$id | title=$title | work_type=$workType | salary=$salary")
    }
}
```

Figure 7: Salary-focused filtering block code snippet

job_id	*title*	*work_type*	*salary*
2914254129	director of operations	FULL-TIME	$57,500
2920450495	service coordinator	FULL-TIME	$65,000
2974397965	marketing coordinator	FULL-TIME	$80,000

Table 3: Sample of full-time postings with a known salary after applying the filter.

This block reduced the dataset from 122,129 rows to 28,749. This shows that a large share of postings in the dataset either do not disclose salary information or are not full-time roles. The

filtered RDD was cached because it is reused in later salary-based tasks. This block answers the question of which postings are suitable for fair salary comparison.

2. *Task 2: Derive salary tiers from normalized salary*

- Input: Filtered full-time salary RDD (rdd_fulltime_salary).
- Operations: flatMap, collect, map, take, reduceByKey, sortBy
- Goal: Assign each full-time salaried posting to a salary tier (LOW / MID / HIGH) using approximate percentile thresholds to derive an intermediate feature from the normalized salary.

```
// 1. Calculate approximate percentile thresholds
val p33 = salaryValues(math.min((totalSalaryRows * 0.33).toInt, totalSalaryRows - 1))
val p67 = salaryValues(math.min((totalSalaryRows * 0.67).toInt, totalSalaryRows - 1))

// 2. Map continuous salary data into discrete analytical tiers
val rdd_salary_bucket = rdd_fulltime_salary.map { row =>
  val jobId    = row.getAs[Any]("job_id")
  val title    = Option(row.getAs[String]("title_clean")).getOrElse("unknown")
  val location = Option(row.getAs[String]("state")).getOrElse("UNKNOWN")
  val salary   = Option(row.getAs[Any]("normalized_salary")).map(_.toString.toDouble).getOrElse(0.0)

  val bucket =
    if (salary < p33) "LOW"
    else if (salary < p67) "MID"
    else "HIGH"

  (jobId, title, location, salary, bucket)
}
```

Figure 8: Salary tier derivation block code snippet

Bucket	*Postings*	*Percentage*
LOW	9,487	33.0%
MID	9,772	34.0%
HIGH	9,490	33.0%

Table 4: Salary tier distribution across full-time postings.

This task converts raw salary values into broader salary categories based on calculated 33rd ($63,741) and 67th ($110,500) percentile thresholds. The table shows a balanced distribution of job postings across the three tiers. This categorization simplifies continuous salary data into analytical groups, making compensation patterns easier to interpret and creating a useful derived feature for later comparisons.

3. *Task 3: Job title keyword frequency analysis*

- Input: Full preprocessed RDD.
- Operations: filter, flatMap, reduceByKey, sortBy, take, reduce
- Goal: Split cleaned job titles into individual words, remove simple stop words and non-letter characters, and count how often each keyword appears.

```
val rdd_title_words = {
  rdd
    .filter(row => Option(row.getAs[String]("title_clean")).exists(_.trim.nonEmpty))
    .flatMap { row =>
      val title = row.getAs[String]("title_clean").trim
      title
        .split("\\s+")
        .map(_.toLowerCase.replaceAll("[^a-z]", ""))
        .filter(word => word.length > 2 && !stopWords.contains(word))
        .map(word => (word, 1))
    }
}

val topTitleWords = {
  rdd_title_words
    .reduceByKey(_ + _)
    .sortBy(_._2, ascending = false)
    .take(20)
}

println("\nTop 20 job title keywords:")
topTitleWords.zipWithIndex.foreach { case ((word, count), i) =>
  println(f"  ${i + 1}%2d. $word%-25s : $count%,d occurrences")
}

val totalKeywordOccurrences = rdd_title_words.map(_._2).reduce(_ + _)
println(s"\nTotal keyword occurrences across all titles: $totalKeywordOccurrences")
```

Figure 9: Job title keyword analysis block code snippet

Rank	*Keyword*	*Occurrences*
1	manager	17,832
2	engineer	9,782
3	sales	8,086
4	senior	7,953
5	assistant	6,641

Table 5: Top 5 most common words found in job titles.

This block shows that **manager** is the most common keyword in job titles, followed by **engineer** and **sales**. This suggests that management, technical, and sales-oriented roles are the most visible in the dataset. It answers the question of what kinds of roles dominate the postings and reveals the strongest hiring patterns in job-title language.

4. *Task 4: Location-level posting and salary aggregation*

- Input: Full preprocessed RDD.
- Operations: filter, map, reduceByKey, map, cache, sortBy, take
- Goal: Group all postings by location and calculate the total number of postings, total applications, and average salary for each location.

```
val rdd_location_raw = {
  rdd
    .filter(row => Option(row.getAs[String]("state")).exists(_.trim.nonEmpty))
    .map { row =>
      val location   = row.getAs[String]("state").trim.toUpperCase
      val applies    = Option(row.getAs[Any]("applies")).map(_.toString.toDouble).getOrElse(0.0)
      val salaryOpt  = Option(row.getAs[Any]("normalized_salary")).map(_.toString.toDouble)
      val salaryVal  = salaryOpt.getOrElse(0.0)
      val salaryFlag = if (salaryOpt.isDefined) 1 else 0

      (location, (applies, salaryVal, salaryFlag, 1))
    }
}

val rdd_location_agg = {
  rdd_location_raw
    .reduceByKey { case ((a1, s1, sc1, pc1), (a2, s2, sc2, pc2)) =>
      (a1 + a2, s1 + s2, sc1 + sc2, pc1 + pc2)
    }
    .map { case (location, (totalApplies, totalSalary, salaryCount, postingCount)) =>
      val avgSalary = if (salaryCount > 0) totalSalary / salaryCount else 0.0
      (location, totalApplies, avgSalary, postingCount)
    }
    .cache()
}
```

Figure 10: Location-level aggregation block code snippet

State	*Job Postings*	*Total Applications*	*Avg Salary*
CA	11,347	19,655	$330,423
TX	10,155	14,311	$658,555
NY	5,955	11,578	$143,200
FL	5,818	5,623	$249,975

Table 6: Aggregated job market statistics per state.

This block answers the question of how job activity and compensation vary across locations. California and Texas have the highest number of job postings in the sample shown, indicating that they are major hiring markets in the dataset. The results highlight geographic differences in both hiring volume and average salary. The aggregated RDD was cached because it is reused again in the salary-ranking task.

5. *Task 5: Location average salary ranking*

- Input: Location-aggregated RDD (rdd_location_agg).
- Operations: filter, map, sortByKey, take, collect
- Goal: Sort locations from highest to lowest average salary to identify higher-paying and lower-paying job markets, filtering for locations with a minimum of 50 postings for reliability.

```
// 1. Filter for reliability, map to (key, value) pairs, and sort by salary descending
val rdd_sorted_by_salary = rdd_location_agg
  .filter { case (_, _, avgSalary, postingCount) => avgSalary > 0 && postingCount >= 50 }
  .map { case (location, applies, avgSalary, postingCount) =>
    (avgSalary, (location, applies, postingCount))
  }
  .sortByKey(ascending = false)

// 2. Action to retrieve the Top 20 highest-paying locations
val top20Locations = rdd_sorted_by_salary.take(20)

// 3. Action to retrieve the Bottom 10 lowest-paying locations
// Requires collect() to bring data to the driver for local extraction
val bottom10Locations = rdd_sorted_by_salary.collect().takeRight(10).reverse
```

Figure 11: Salary tier distribution across full-time postings.

rank	*location*	*avg salary*	*postings*
1	KY	$750,234	1,172
2	SC	$665,633	1,533
3	TX	$656,746	10,197
4	OK	$599,642	790
5	MN	$361,021	1,816

Table 7: Location average salary ranking

This block answers the question of which geographic markets offer the highest and lowest average compensation. By sorting the previously aggregated location data and filtering out regions

with fewer than 50 postings, the ranking ensures that the averages are based on a reliable sample size. The results highlight significant geographic differences in compensation, identifying states like Kentucky, South Carolina, and Texas as the highest-paying job markets within the dataset.

4.3 Reflection on RDD Usage

To improve performance, two RDDs were cached using .cache(). The first was rdd_fulltime_salary in Task 1, since this filtered subset serves as the input for Task 2, where it is reused for salary extraction, salary bucketing, and bucket distribution. The second was rdd_location_agg in Task 4, as its results are consumed again in Task 5 for salary ranking. Without caching, Spark would recompute the full lineage from the parquet file each time these RDDs were reused. One structural challenge encountered during development was that Spark's interactive shell does not handle long multi-line transformation chains smoothly when using :load; long pipelines often had to be wrapped in braces or split into named intermediate values to avoid parsing or chaining issues.

The main scalability limitation is the use of collect() in Tasks 2 and 5, which pulls data to the driver to perform local sorting and percentile calculations. This is acceptable at the current dataset size of roughly 28,749 salary records, but it would become a memory bottleneck if the dataset scaled to tens of millions of records.

5 SQL Operations

5.1 DataFrame and View Creation

After completing the preprocessing phase, the cleaned dataset was loaded into Apache Spark as a DataFrame and registered as a temporary SQL view. This setup allowed us to execute SQL queries directly on the dataset using Spark SQL. The DataFrame provides a structured representation of the data, while the temporary view enables the use of standard SQL syntax for analysis.

```
// Load the preprocessed dataset from Phase 2.
val df = spark.read.parquet("data/preprocessed_dataset.parquet")

// Register the DataFrame as a temporary view so we can query it using SQL.
df.createOrReplaceTempView("job_postings")
println(s"Total rows loaded: ${df.count()}")
```

Figure 12: DataFrame and View Creation

5.2 Key Analytical Queries

Query 1: Average Salary by Work Type and Pay Period

Question: How does salary vary across different employment types and pay periods?

This query calculates average, minimum, and maximum salaries for each combination of work type and pay period. The results show that full-time yearly roles provide the most consistent and realistic salary values. In contrast, hourly roles, especially internships and contract jobs, show unusually high averages due to outliers.

This indicates that salary patterns depend on how compensation is reported. Full-time yearly salaries better reflect real market trends, while hourly values require careful interpretation. This supports the project objective of analyzing how job attributes affect salary levels.

```
// ============================================================
// Q1: Average Salary by Work Type and Pay Period
// Question: What is the average salary for each combination
//           of employment type and pay period?
// Techniques: GROUP BY, HAVING, aggregation functions
// ============================================================

println("\n==== Q1: Average Salary by Work Type and Pay Period ====")

spark.sql("""
  SELECT
    work_type_std,
    pay_period_std,
    COUNT(*)                          AS total_postings,
    ROUND(AVG(normalized_salary), 2)  AS avg_salary,
    ROUND(MIN(normalized_salary), 2)  AS min_salary,
    ROUND(MAX(normalized_salary), 2)  AS max_salary
  FROM job_postings
  WHERE normalized_salary IS NOT NULL
    AND work_type_std     IS NOT NULL
    AND pay_period_std    IS NOT NULL
  GROUP BY work_type_std, pay_period_std
  HAVING COUNT(*) > 50
  ORDER BY avg_salary DESC
""").show()
```

Figure 13: Query 1 Code

```
==== Q1: Average Salary by Work Type and Pay Period ====
+-------------+--------------+--------------+----------+----------+-----------+
|work_type_std|pay_period_std|total_postings|avg_salary|min_salary| max_salary|
+-------------+--------------+--------------+----------+----------+-----------+
|   INTERNSHIP|        HOURLY|           200|1763954.56|   24960.0|4.7179392E7|
|    FULL-TIME|        HOURLY|          8723| 453601.89|   14560.0|    5.356E8|
|    FULL-TIME|        YEARLY|         19556| 116334.49|       0.0|   950000.0|
|    FULL-TIME|        WEEKLY|            56| 112250.89|   70772.0|  175686.94|
|    PART-TIME|        WEEKLY|            80| 108639.62|   50804.0|   210548.0|
|     CONTRACT|        HOURLY|          3313| 104731.06|   22880.0|  5314400.0|
|    PART-TIME|        HOURLY|          1899|  92104.74|   22360.0|   6.4896E7|
|    TEMPORARY|        YEARLY|            53|  81728.27|      30.0|   212500.0|
|    PART-TIME|        YEARLY|           235|  81414.66|      13.0|   475000.0|
|     CONTRACT|        YEARLY|           388|  78383.39|       0.0|   596000.0|
|    TEMPORARY|        HOURLY|           327|  70525.39|   16224.0|   468000.0|
|    FULL-TIME|       MONTHLY|           405|  65376.36|     420.0|  1200000.0|
|        OTHER|        HOURLY|            87|  59262.79|   27040.0|   520000.0|
|     CONTRACT|       MONTHLY|            67|  49231.72|     510.0|   144000.0|
+-------------+--------------+--------------+----------+----------+-----------+
```

Figure 14: Query 1 Output

Query 2: Top 10 States by Average Salary

Question: Which locations offer the highest salaries, and how do they compare in terms of job availability?

This query ranks states based on average salary while also considering job postings and application counts. The results show that some states, such as KY and TX, have higher average salaries, while highly active states like CA have more job postings but not the highest salaries.

This suggests that salary is not directly tied to job volume. Larger markets may have more competition, while smaller markets may offer higher pay. This confirms that location is an important factor influencing salary distribution.

```
// ============================================================
// Q2: Top 10 States by Average Salary
// Question: Which states offer the highest average salaries,
//           and how many job postings do they have?
// Techniques: GROUP BY, HAVING, ORDER BY, LIMIT, PERCENTILE
// ============================================================

println("\n==== Q2: Top 10 States by Average Salary ====")

spark.sql("""
  SELECT
    state,
    COUNT(*)                                          AS total_postings,
    ROUND(AVG(normalized_salary), 2)                  AS avg_salary,
    ROUND(PERCENTILE_APPROX(normalized_salary, 0.5), 2)  AS median_salary,
    CAST(SUM(applies) AS BIGINT)                      AS total_applications
  FROM job_postings
  WHERE normalized_salary IS NOT NULL
    AND state IS NOT NULL
  GROUP BY state
  HAVING COUNT(*) >= 100
  ORDER BY avg_salary DESC
  LIMIT 10
""").show()
```

Figure 15: Query 2 Code

```
==== Q2: Top 10 States by Average Salary ====
+-----+--------------+----------+-------------+------------------+
|state|total_postings|avg_salary|median_salary|total_applications|
+-----+--------------+----------+-------------+------------------+
|   KY|           281| 750234.13|      59280.0|               158|
|   SC|           262| 665632.65|      62400.0|               119|
|   TX|          1954| 656746.15|      77500.0|              4495|
|   OK|           170| 599641.63|      60000.0|               129|
|   MN|           447| 361020.71|      67600.0|               284|
|   CA|          5991|  337262.4|      92500.0|             11905|
|   FL|          1369| 249595.69|      66508.0|              2203|
|   NC|           836| 207135.28|      78000.0|              1212|
|   WA|          1347| 183123.02|      91280.5|              2390|
|   NY|          3010| 142192.73|      87500.0|              6964|
+-----+--------------+----------+-------------+------------------+
```

Figure 16: Query 2 Output

Query 3: Salary by Experience Level vs Overall Average

Question: How does salary differ across experience levels compared to the overall average?

This query compares the average salary for each experience level with the overall dataset average. The results show that most roles fall below the overall average, while internship roles appear unusually high due to data irregularities.

This highlights that experience level influences salary, but the dataset is affected by extreme values. This insight is important for the machine learning phase, as it shows the need to handle outliers carefully when predicting salaries.

```
// ============================================================
// Q3: Salary by Experience Level vs Overall Average
// Question: How does the average salary at each experience
//           level compare to the overall average salary?
// Techniques: GROUP BY, Window Function (AVG OVER)
// ============================================================

println("\n==== Q3: Salary by Experience Level vs Overall Average ====")

spark.sql("""
  SELECT
    formatted_experience_level,
    COUNT(*)                                      AS postings_count,
    ROUND(AVG(normalized_salary), 2)              AS avg_salary,
    ROUND(AVG(AVG(normalized_salary)) OVER (), 2)  AS overall_avg_salary,
    ROUND(
      AVG(normalized_salary) -
      AVG(AVG(normalized_salary)) OVER (), 2
    )                                             AS diff_from_overall
  FROM job_postings
  WHERE normalized_salary          IS NOT NULL
    AND formatted_experience_level IS NOT NULL
  GROUP BY formatted_experience_level
  ORDER BY avg_salary DESC
""").show()
```

Figure 17: Query 3 Code

```
+--------------------------+--------------+----------+------------------+-----------------+
|formatted_experience_level|postings_count|avg_salary|overall_avg_salary|diff_from_overall|
+--------------------------+--------------+----------+------------------+-----------------+
|                Internship|           378| 963207.07|         314493.62|        648713.45|
|              Entry level|          9115| 247214.72|         314493.62|        -67278.91|
|         Mid-Senior level|         12864| 220845.14|         314493.62|        -93648.48|
|                 Executive|           377| 201434.24|         314493.62|       -113059.38|
|                  Director|          1262| 172341.66|         314493.62|       -142151.97|
|                 Associate|          3859|  81918.91|         314493.62|       -232574.71|
+--------------------------+--------------+----------+------------------+-----------------+
```

Figure 18: Query 3 Output

Query 4: Most Common Job Titles and Their Salaries

Question: What are the most common job titles, and how do their salaries compare?

This query analyzes the most frequent job titles along with their average salaries. The results show that roles such as project manager and sales manager are common, but higher frequency does not necessarily mean higher salary.

This indicates that job title plays a key role in both demand and compensation. Some roles are highly demanded but lower paying, while others are less frequent but offer higher salaries. This makes job title an important feature for salary prediction.

```
// ============================================================
// Q4: Top 15 Most Common Job Titles and Their Salaries
// Question: What are the most frequently posted job titles,
//           and how is the average salary for each?
// Techniques: CTE (WITH), Window Function (RANK OVER),
//             GROUP BY, HAVING, COUNT DISTINCT
// ============================================================

print("\n==== Q4: Top 15 Most Common Job Titles and Their Salaries ====")

spark.sql("""
  WITH title_stats AS (
    SELECT
      title_clean,
      COUNT(*)                              AS posting_count,
      ROUND(AVG(normalized_salary), 2)      AS avg_salary,
      COUNT(DISTINCT company_name)          AS distinct_companies
    FROM top_postings
    WHERE title_clean IS NOT NULL
    GROUP BY title_clean
    HAVING COUNT(*) >= 30
  )
  SELECT *,
    RANK() OVER (ORDER BY posting_count DESC) AS popularity_rank
  FROM title_stats
  ORDER BY popularity_rank
  LIMIT 15
""").show()
```

Figure 19: Query 4 Code

```
+--------------------+-------------+----------+------------------+---------------+
|         title_clean|posting_count|avg_salary|distinct_companies|popularity_rank|
+--------------------+-------------+----------+------------------+---------------+
|       sales manager|          669| 118401.06|                63|              1|
|customer service ...|          473|  40889.19|               153|              2|
|     project manager|          350| 110955.79|               303|              3|
|assistant store m...|          320|  45719.08|                52|              4|
|   senior accountant|          241|  88408.91|               187|              5|
|administrative as...|          238|  51506.57|               196|              6|
| executive assistant|          222|   83343.7|               156|              7|
|    registered nurse|          210|  94347.62|                94|              8|
|         salesperson|          204|   65392.0|                70|              9|
|    staff accountant|          200|  67139.42|               163|             10|
|        receptionist|          200|  43096.04|                52|             10|
|   account executive|          194|  79173.08|               156|             12|
|retail sales asso...|          190|  35292.12|                18|             13|
|     sales associate|          190| 2751503.1|                37|             13|
|       store manager|          185|  69395.94|                73|             15|
+--------------------+-------------+----------+------------------+---------------+
```

Figure 20: Query 4 Output

Query 5: Job Engagement Analysis by Work Type

Question: How does job engagement vary across different employment types?

This query examines engagement metrics such as applications, views, and application rate across work types. The results show that contract roles receive the highest engagement, while full-time roles show moderate engagement, and part-time or volunteer roles receive the least.

This suggests that job type and flexibility influence user interest. Engagement metrics provide additional insight into job attractiveness and complement salary analysis by reflecting candidate behaviour.

```
// ============================================================
// Q5: Job Engagement Analysis by Work Type
// Question: Which employment type attracts the most applicants,
//           and what is the application-to-view conversion rate?
// Techniques: GROUP BY, aggregation functions, derived metric
// ============================================================

println("\n==== Q5: Job Engagement Analysis by Work Type ====")

spark.sql("""
  SELECT
    work_type_std                                    AS work_type,
    COUNT(*)                                         AS total_postings,
    ROUND(AVG(normalized_salary), 2)                AS avg_salary,
    ROUND(AVG(applies), 2)                           AS avg_applications,
    ROUND(AVG(views), 2)                             AS avg_views,
    ROUND(
      AVG(applies) / NULLIF(AVG(views), 0)
      * 100, 2)                                      AS application_rate_pct
  FROM job_postings
  WHERE work_type_std IS NOT NULL
  GROUP BY work_type_std
  ORDER BY avg_applications DESC
""").show()
```

Figure 21: Query 5 Code

```
==== Q5: Job Engagement Analysis by Work Type ====
+----------+-------------+----------+----------------+---------+--------------------+
| work_type|total_postings|avg_salary|avg_applications|avg_views|application_rate_pct|
+----------+-------------+----------+----------------+---------+--------------------+
|  CONTRACT|        11955| 101183.71|            6.76|    31.52|               21.45|
|INTERNSHIP|          955|1457949.55|            4.81|    23.07|               20.83|
|     OTHER|          449|  78173.63|            1.63|    10.66|               15.29|
| FULL-TIME|        97551| 218296.57|            1.52|    13.01|                11.7|
| TEMPORARY|         1176|  72425.35|             1.4|    10.73|                13.1|
| PART-TIME|         9488|  91326.23|             0.5|     7.8|                6.46|
| VOLUNTEER|          555|  17545.63|            0.39|     7.46|                5.21|
+----------+-------------+----------+----------------+---------+--------------------+
```

Figure 22: Query 5 Output

5.3 Summary and Insights from SQL Analysis

The SQL analysis shows that salary levels are influenced by several key factors, including employment type, location, experience level, and job title. Full-time yearly roles provide the most stable salary patterns, while other pay structures introduce variability. Location also plays an important role, as some states offer higher salaries despite having fewer job postings.

In addition, experience level and job title were identified as strong indicators of salary variation, although the presence of outliers affects the overall distribution. Engagement metrics further highlight differences in job attractiveness, with contract roles receiving higher interaction from candidates.

Overall, these findings provide a clearer understanding of labor market patterns and directly support the project objective of analyzing salary variation.

6 Machine Learning Operations

6.1 Problem Formulation

Type: Regression

Target Variable: *log_salary*, the natural log of the annual salary. Log-transform reduces skew, stabilizes variance, and limits the impact of extreme salaries.

Input Feature:

Feature	*Reasoning*
Engagement features (`log_views, log_applies, engagement`)	Capture candidate interest and posting visibility.
Title-based features (`title_len, title_words`)	Reflect title complexity and possible seniority level.
Temporal features (`listed_month, listed_dow, listed_hour`)	Capture posting time patterns that may relate to salary.
Remote-work feature (`is_remote`)	Distinguishes remote and on-site roles, which may differ in pay.
Categorical labor-market features (`work_type_std, formatted_experience_level, state`)	Represent job type, experience level, and location, which strongly affect salary.

Text feature (`title_clean` via TF-IDF)	Captures job title keywords that help distinguish salary differences across roles.

Table 8: Input Features

Constraints:

- **Noisy labels:** Salary values may be inconsistent or estimated, which adds noise to the target.
- **Class sparsity:** The `state` column has 297 distinct values, creating sparse one-hot vectors.
- **Skewed target:** Most salary values are in the lower to middle range, while a few very high salaries create a long right tail. Log transformation was used to reduce this skewness.
- **Sparse TF-IDF vectors:** Since each title uses only a few words, most TF-IDF values are zero. `withMean = false` was used to keep the vectors sparse and memory-efficient.

Only rows with `pay_period_std = "YEARLY"` and salary in [10,000–1,000,000] were retained, yielding a focused and comparable set for regression.

6.2 Feature Pipeline

The pipeline was constructed manually (without Spark ML Pipeline object) in the following sequential stages, all fitted on training data only to prevent data leakage:

1. **Categorical Encoding**
   - StringIndexer was applied to work_type_std, formatted_experience_level, and state to convert categorical values into numeric indices. These indexed columns were then transformed using OneHotEncoder, producing sparse vectors for employment type, experience level, and location.
2. **Text Feature Extraction (TF-IDF)**
   - The title_clean column was processed using TF-IDF. Tokenizer split each title into words, StopWordsRemover removed common terms, HashingTF converted the tokens into term-frequency vectors with 500 features, and IDF reweighted them to produce the final title_tfidf representation.

3. **Feature Assembly**
   - VectorAssembler combines all numeric, encoded, and TF-IDF features into a single features_raw vector. handleInvalid="skip" drops any remaining null rows.
4. **Scaling**
   - StandardScaler was applied with withStd = true and withMean = false to standardize the final feature vector. Mean-centering was disabled because TF-IDF vectors are sparse, and centering them would make them dense and memory-intensive.

Excluded features: job_id, company_id, raw title, description (free text, not engineered), and normalized_salary (would cause target leakage).

6.3 Model Choice

Model	Rationale	Key Hyperparameters
Linear Regression	Interpretable baseline; captures linear salary relationships	maxIter=200, regParam=0.05, elasticNetParam=0.1
Random Forest	Handles non-linear interactions and categorical features robustly; resistant to overfitting via bagging.	numTrees=100, maxDepth=10, seed=42
Gradient Boosted Trees (GBT)	Best at capturing complex feature interactions sequentially; typically outperforms RF on structured tabular data.	maxIter=200, maxDepth=10, stepSize=0.05, subsamplingRate=0.8, seed=42

Table 9: Model Choice

A conservative stepSize=0.05 and subsamplingRate=0.8 were used for GBT to reduce overfitting. No automated hyperparameter tuning (CrossValidator) was performed due to computational constraints.

6.4 Training and Evaluation Setup

Train/Test Split: The dataset was split 70% training / 30% testing using a fixed random seed of 42 to ensure reproducibility. After filtering for yearly salaries, the split produced approximately 24,500 training rows and 10,500 test rows.

Class Imbalance: Not applicable, as this is a regression task. However, low-salary postings are underrepresented after filtering for yearly pay periods. This causes the model to overestimate salaries at the lower end, a symptom of regression toward the mean rather than class imbalance.

Evaluation Metrics: Three metrics were used to evaluate all models on the table below:

Metric	Why it was chosen
RMSE	Penalizes large prediction errors more heavily; appropriate given the presence of salary outliers
MAE	More robust to outliers; gives the average absolute error in log-salary units, which is easier to interpret
R^2	Measures how much salary variance the model explains compared to simply predicting the mean

Table 10: Evaluation Metrics

All metrics were computed on the held-out test set only. The target variable log_salary is on a log scale, so an RMSE of ~0.37 corresponds to roughly ±45% error in the original salary scale, reasonable given that salary data in job postings is self-reported and noisy.

6.5 Results

```
============================================================
MODEL COMPARISON SUMMARY
============================================================
Baseline          -> RMSE: 0.4788 | MAE: 0.3814 | R2: -0.0000
Linear Regression -> RMSE: 0.3787 | MAE: 0.2841 | R2: 0.3745
Random Forest     -> RMSE: 0.4009 | MAE: 0.3075 | R2: 0.2989
GBT               -> RMSE: 0.3754 | MAE: 0.2744 | R2: 0.3852

Best by RMSE: GBT
Best by MAE : GBT
Best by R2  : GBT
```

Figure 23: Model Comparison Summary

The following figure summarizes the performance of all models on the test set. The target variable is log_salary (log-transformed annual salary).

All models significantly outperformed the baseline, which predicts the training mean for every observation. The Gradient Boosted Trees (GBT) model achieved the best performance across all three metrics, with an RMSE of 0.3754, MAE of 0.2744, and R^2 of 0.3852. An RMSE

of 0.3754 in log-salary units corresponds to an approximate 45% relative error in the original salary scale, which is reasonable given the inherent noise in self-reported job posting salary data.

6.6 Interpretation

6.6.1 Feature Importance

Random Forest - Top Features:

Feature	*Importance*	*Type*
encoded_feature_11	0.1317	One-Hot (state/experience)
encoded_feature_9	0.1041	One-Hot (state/experience)
encoded_feature_12	0.0766	One-Hot (state/experience)
encoded_feature_302	0.0478	TF-IDF (job title keyword)
title_words	0.0450	Title word count

Table 11: Random Forest - Top 5 Feature Importances

GBT - Top Features:

Feature	*Importance*	*Type*
title_len	0.0636	Title character length
listed_hour	0.0426	Hour of posting
log_views	0.0418	Posting engagement
title_words	0.0294	Title word count
listed_dow	0.0202	Day of week

Table 12: GBT - Top 5 Feature Importances

6.6.2 Domain Interpretation:

Both models agree that job title characteristics (length and word count) are among the strongest predictors. This aligns with domain expectations, longer and more specific titles such as "Senior Staff Machine Learning Engineer" typically correspond to higher-seniority roles and higher salaries.

The Random Forest places higher importance on encoded categorical features (state and experience level), suggesting that geographic location and seniority are strong salary signals when captured as categorical variables.

The presence of TF-IDF features (encoded_feature_302, encoded_feature_307) in the top importances confirms that the semantic content of job titles contributes meaningful salary signal beyond simple length metrics.

Interestingly, listed_hour appears prominently in GBT. While this may seem counterintuitive, it could reflect posting patterns of different company types, for example, international companies posting at different local hours.

```
=== RF Feature Importances (Top 15) ===
encoded_feature_11            -> 0.131712
encoded_feature_9             -> 0.104109
encoded_feature_12            -> 0.076623
encoded_feature_7             -> 0.073140
encoded_feature_302           -> 0.047791
title_words                   -> 0.045005
encoded_feature_307           -> 0.035609
encoded_feature_14            -> 0.032378
title_len                     -> 0.028913
encoded_feature_10            -> 0.026519
encoded_feature_516           -> 0.021819
encoded_feature_8             -> 0.019734
encoded_feature_563           -> 0.016797
listed_hour                   -> 0.015686
encoded_feature_380           -> 0.014883

=== GBT Feature Importances (Top 15) ===
title_len                     -> 0.063568
listed_hour                   -> 0.042588
log_views                     -> 0.041806
title_words                   -> 0.029355
listed_dow                    -> 0.020245
encoded_feature_9             -> 0.015085
encoded_feature_8             -> 0.013137
encoded_feature_15            -> 0.012520
encoded_feature_7             -> 0.012194
engagement                    -> 0.010785
encoded_feature_315           -> 0.010699
log_applies                   -> 0.010687
encoded_feature_14            -> 0.010619
encoded_feature_11            -> 0.010610
encoded_feature_565           -> 0.009684
```

Figure 24: Top 15 Feature Importances for Random Forest and GBT Models

6.6.3 Error Analysis

Looking at the GBT back-transformed predictions:

Actual Salary	*Predicted Salary*	*Error*
$12,700	$151,878	Very high (extreme undervaluation)
$31,500	$53,031	Moderate overestimation
$40,000	$97,376	High overestimation

Table 13: Sample Error Analysis - GBT Predicted vs Actual Salary

The model consistently overestimates low salaries and pulls predictions toward the training mean. This is a classic symptom of regression toward the mean, the model performs well in the middle salary range but fails at the extremes.

Possible causes:

- Low-salary postings are underrepresented in the training data after filtering for yearly salaries
- The model lacks direct features distinguishing entry-level from senior roles in all cases
- Some job postings report salary ranges rather than exact figures, introducing label noise

```
=== GBT Predictions 🖼 Salary Scale ===
+------------------+------------------+------------------+------------------+
|label             |prediction_gbt    |actual_salary     |predicted_salary  |
+------------------+------------------+------------------+------------------+
|9.44943600950432  |11.930842025187097|12700.0           |151878.39412779396|
|10.12667110305036 |12.327949799104926|25000.000000000007|225922.064810371  |
|10.263711192398603|11.424352793396084|28671.999999999978|91522.65966349925 |
|10.33601563244398 |11.105451733042713|30821.97499999998 |66531.8928076997  |
|10.357774570341576|10.878666394341954|31499.999999999996|53031.827858293545|
|10.373522431293592|11.133639802567973|31999.999999999993|68434.0091649942  |
|10.378198833381107|10.600774095264807|32150.000000000025|40164.91764645669 |
|10.505094936455558|11.261984404267709|36499.99999999998 |77805.82290023507 |
|10.530521639636548|11.113880884289397|37440.000000000015|67095.07887979141 |
|10.53212287826962 |11.21104349013963 4|37500.000000000015|73941.53305395586 |
|10.53212287826962 |10.846095502422907|37500.000000000015|51332.32869768973 |
|10.545367754151743|11.098080543858169|38000.000000000015|66043.26934822013 |
|10.545367754151743|11.215673170320976|38000.000000000015|74284.6569979811  |
|10.584587455185398|10.932822906396863|39520.00000000003 |55983.09471321755 |
|10.596659732783579|11.486351739069   |40000.0           |97376.6248657459  |
+------------------+------------------+------------------+------------------+
```

Figure 25: Top 15 Feature Importances for Random Forest and GBT Models

6.7 Limitations

6.7.1 Methodological Limitations

- No hyperparameter tuning was performed. Parameters for GBT (maxIter=200, maxDepth=10, stepSize=0.05) and Random Forest (numTrees=100, maxDepth=10) were set manually. A CrossValidator with ParamGridBuilder could improve results.
- TF-IDF tokenization and HashingTF were applied within the ML pipeline rather than as part of the preprocessing phase (Phase 2). In a production setting, these steps should be part of the data transformation pipeline.
- The StandardScaler used `withMean=false` due to sparse TF-IDF vectors, which means only variance scaling was applied (not full standardization).

6.7.2 Data Limitations

- **Self-reported salary data:** Salary ranges in job postings are set by recruiters and may not reflect actual compensation, introducing significant label noise.
- **Missing contextual features:** Company name, industry sector, and specific skills required are not included in the model. These are strong salary predictors but require additional feature engineering (e.g., company size encoding, skill embeddings).
- **Geographic sparsity:** The `state` feature has 297 distinct values, many with few postings, making One-Hot Encoding produce sparse and less informative vectors.
- **Temporal bias:** The dataset covers 2023–2024 only. Salary trends may shift over time, and the model may not generalize to future postings.

6.7.3 What Would Be Needed to Improve Results

- Adding company-level features such as industry, company size, and funding stage
- Using Word2Vec or BERT embeddings instead of TF-IDF for richer job title representation
- Applying CrossValidator for systematic hyperparameter tuning
- Including full job description text as an additional TF-IDF feature
- Collecting more data for low-salary postings to reduce bias toward the mean

7 Conclusion and Future Work

7.1 Summary of Contributions

This project analyzed LinkedIn job postings to understand salary patterns and predict salary levels using Apache Spark. The work combined data preprocessing, RDD operations, Spark SQL analysis, and machine learning to study how job-related factors affect compensation.

The results showed that salary is influenced by several important factors, especially *location*, *employment type, experience level, and job title*. Full-time yearly jobs gave the most reliable salary patterns, while hourly and mixed pay-period records were harder to compare because of inconsistent salary formats. The analysis also showed that a high number of job postings in a location does not always mean higher salaries.

The machine learning results showed that salary prediction is possible, but it is affected by the quality of the salary data. Many postings did not include salary information, and some salary values were extreme or inconsistent. Among the tested models, **Gradient Boosted Trees** performed the best, but the model still had difficulty predicting very low or very high salaries.

Overall, the project showed how Spark can be used to process, analyze, and model a large real-world job market dataset. It also confirmed that data cleaning and feature preparation are critical steps before building reliable analysis or prediction models.

7.2 Reflection

Through this project, we learned that working with big data requires more than applying analysis tools. A major part of the work was preparing the data, handling missing values, standardizing fields, and creating useful features. Without these steps, the analysis and machine learning results would not be reliable.

We also learned how different Spark tools support different tasks. RDDs gave more control over custom transformations, Spark SQL made it easier to answer analytical questions, and MLlib allowed us to build scalable prediction models.

The main challenges were missing salary values, outliers, and debugging Spark transformations. We also noticed that some operations, such as collecting data to the driver, may work for this dataset but would not be suitable for much larger datasets. This helped us understand the importance of scalability when designing big data workflows.

7.3 Future Work

Future work can improve the project by adding more features from the full LinkedIn dataset, such as company industry, company size, and company location. These features could make salary prediction more accurate because salary often depends on the employer and industry.

Another improvement is to use job descriptions more deeply. Extracting skills, tools, and seniority keywords from descriptions could provide stronger signals for salary prediction than job titles alone.

The machine learning part can also be improved by using hyperparameter tuning and testing more advanced models. In addition, the model should be evaluated separately for low, medium, and high salary ranges to better understand where it performs well and where it fails.

Finally, the project could be developed into a simple dashboard or salary insight tool. Users could enter a job title, location, work type, and experience level to receive an estimated salary range and view related job market trends.

In conclusion, this project provided useful insights into salary patterns in LinkedIn job postings and showed the value of Spark for big data analysis. With richer features and stronger model tuning, the results could become more accurate and more useful for job seekers, recruiters, and educational

8 References

[1] H. Lasi, P. Fettke, H.-G. Kemper, T. Feld, and M. Hoffmann, “Industry 4.0,” *Business & Information Systems Engineering*, vol. 6, no. 4, pp. 239–242, Aug. 2014, doi:10.1007/s12599-014-0334-4.

9 Appendix

Appendix A – Project Links

- **GitHub Repository**:

https://github.com/Jana-Alrzoog/JobAnalytics_BigDataProject.git

- **Google Drive:**

https://drive.google.com/drive/folders/1F_KEEZ8JrIi7R4ANPA0UaIjpAif9pqst?usp=sharing